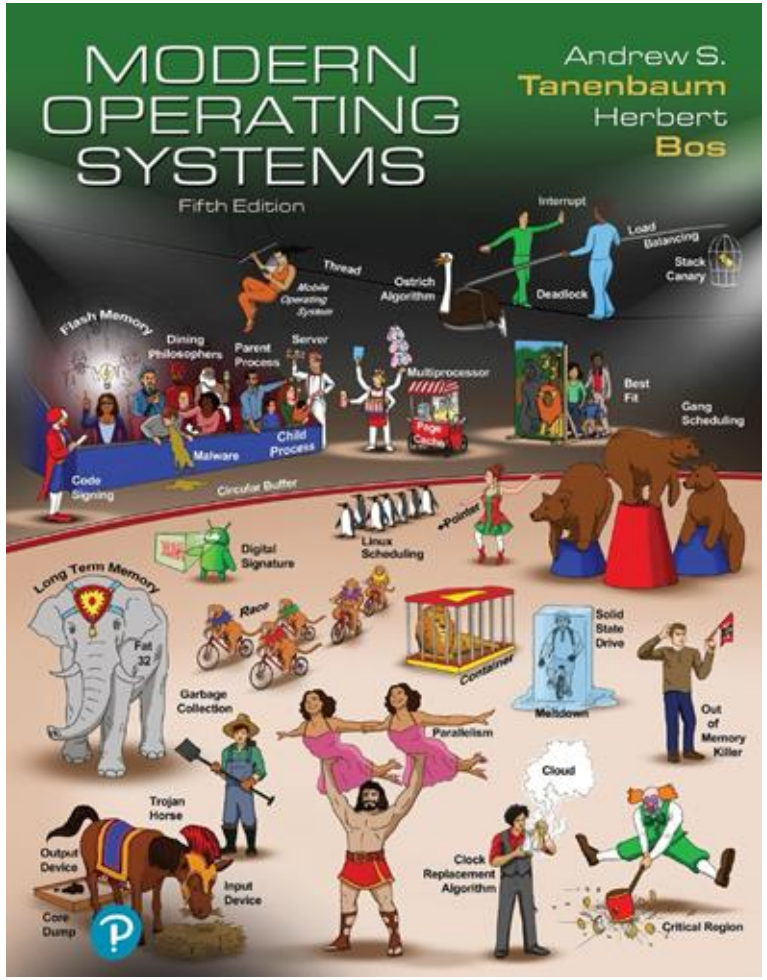
Modern Operating Systems

Fifth Edition



Memory Management

Pearson

Copyright © 2023, 2014, 2008 Pearson Education, Inc. All Rights Reserved

Memory

- Memory Abstractions
- Virtual Memory
- Page Replacement Algorithms
- Design Issues for Paging Systems

Virtual Memory (1)

- There is a need to run programs that are too large to fit in memory
- Solution adopted in the 1960s, split programs into little pieces, called overlays
  - Kept on the disk, swapped in and out of memory
  - Initially, only the overlay manager (O M) was loaded
  - The O M would then load overlay 0 and run it
  - When done, it would tell the O M to load overlay 1, etc.
- Nowadays, we have a better solution: *virtual memory*
  - Often implemented using *paging*

Virtual Memory (2)

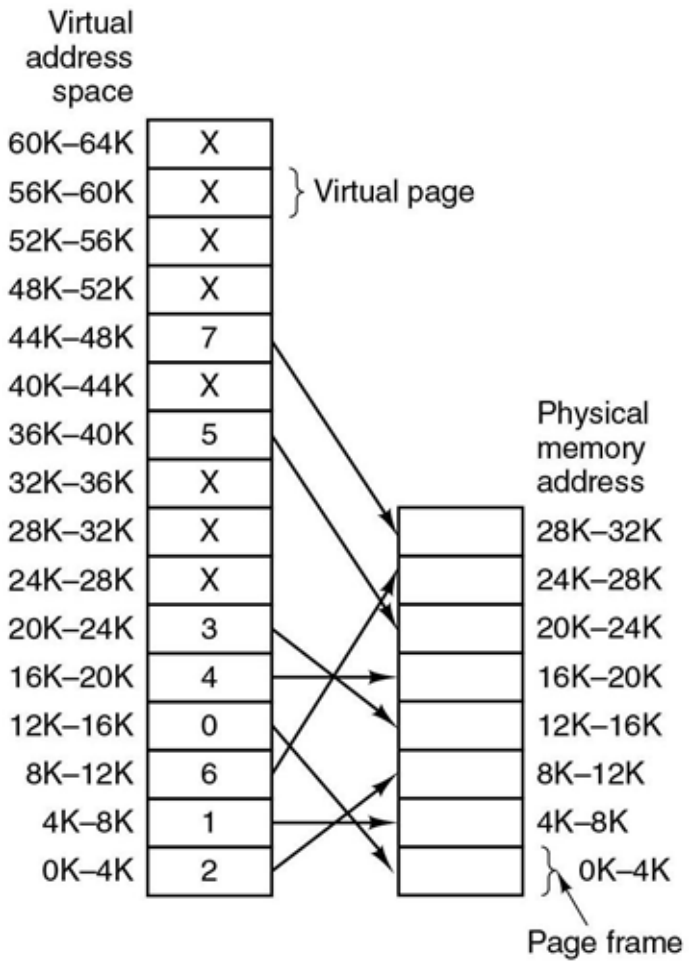
Problem: So far, memory allocation to processes has been restricted to contiguous chunks. This means that a process must be loaded into a single, uninterrupted block of available memory, which can lead to inefficiencies like external fragmentation (wasted space between allocated blocks).

Solution:

- Establish the illusion of a vast address space for the process.
  - e.g., 32-bit or 48-bit (16-bit address space in our example)
- This construct is termed the **virtual address space**.

Practical Issue:

- The actual physical memory is significantly constrained
  - Translate the virtual addresses employed by the process into corresponding physical addresses.
  - This translation is routinely executed by the Memory Management Unit (MMU).



Modern systems use paging: Divide physical and virtual memory into pages of fixed size (e.g. 4096 bytes)

Virtual to Physical Memory Address Translation

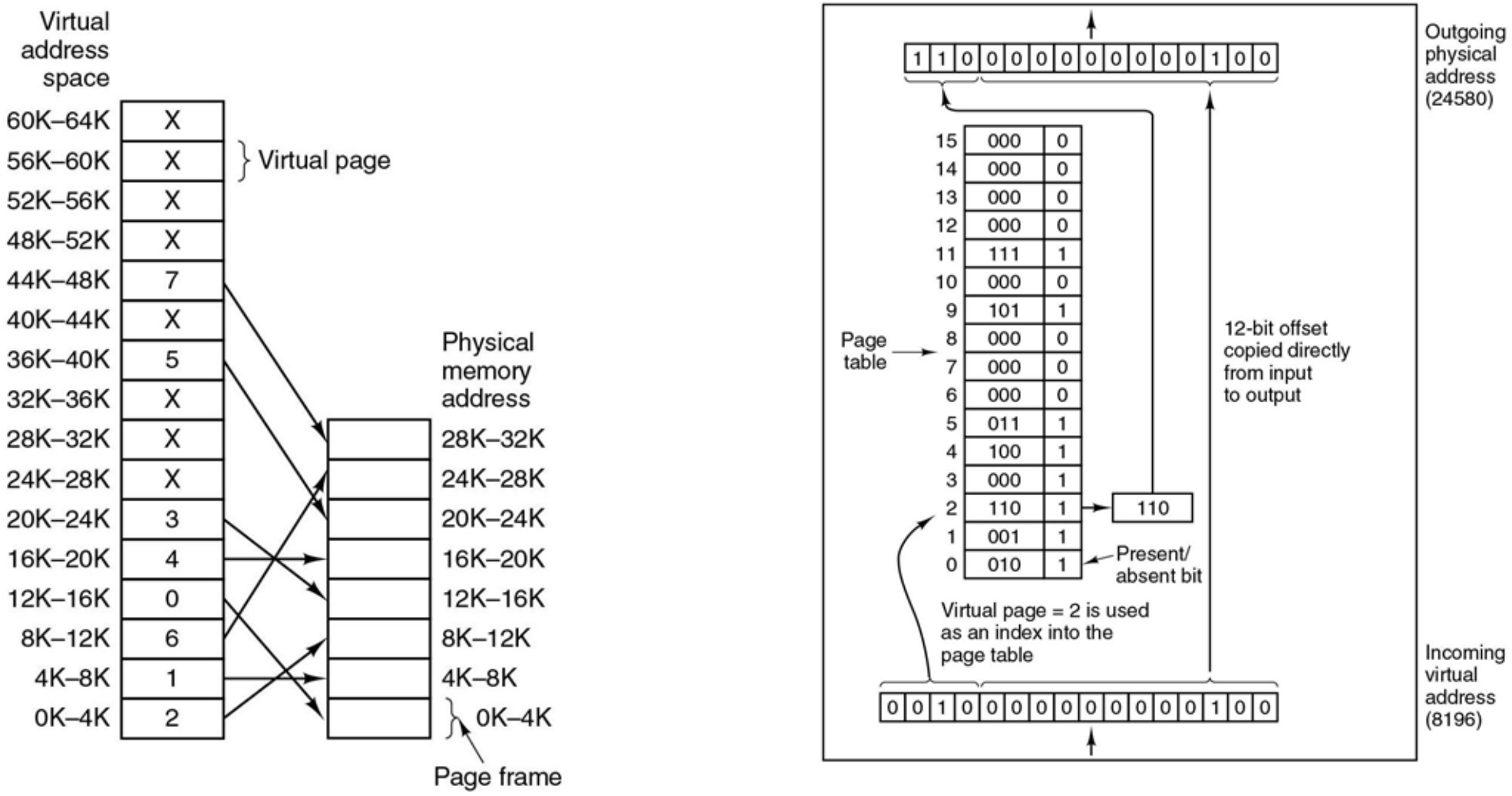
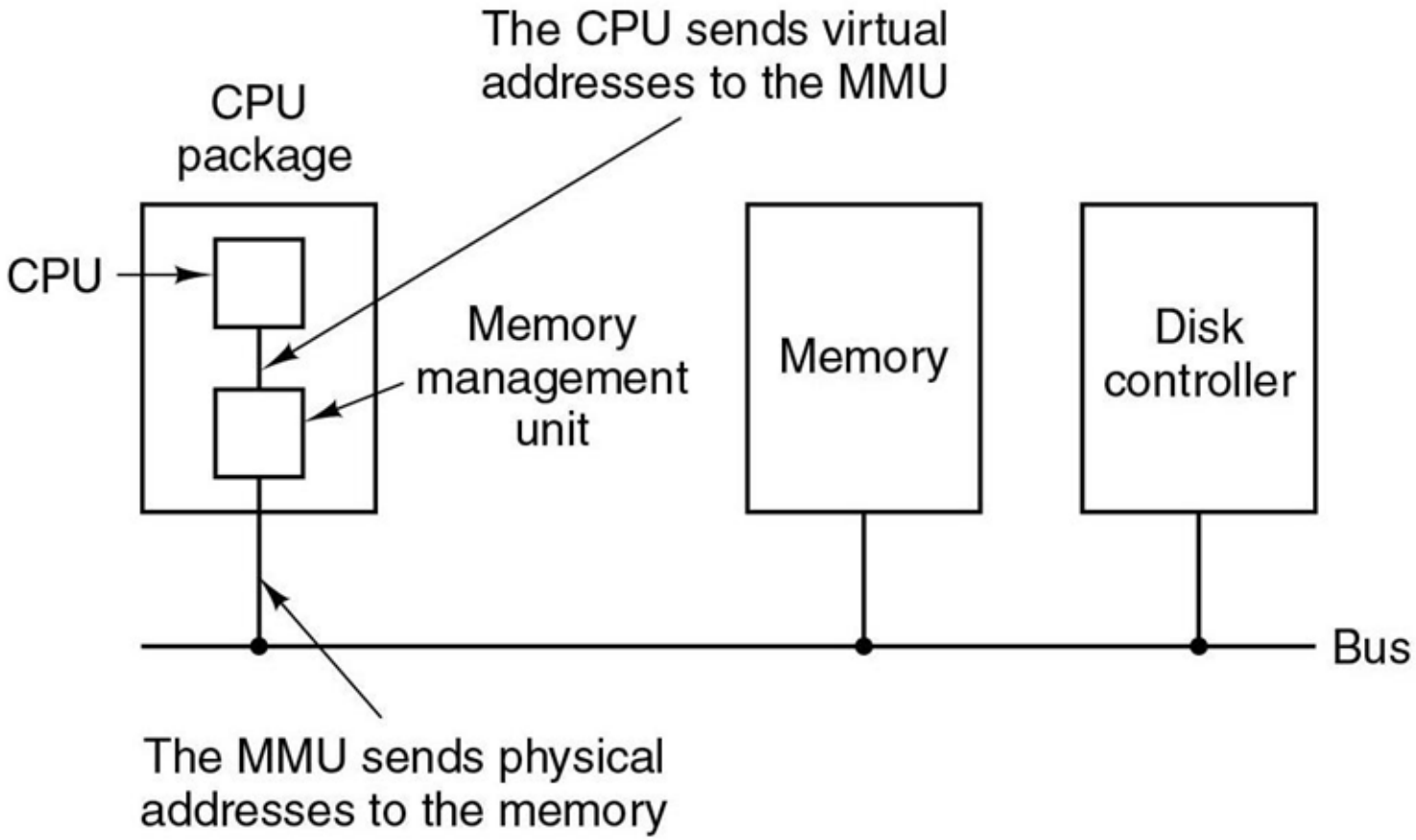


Figure 3.10 The internal operation of the MMU with 16 4-K B pages.

Memory Management Unit



Question: where we store page table of process?

Structure of a Page Table Entry

Questions:

- How big is the page table?
- How many page tables?

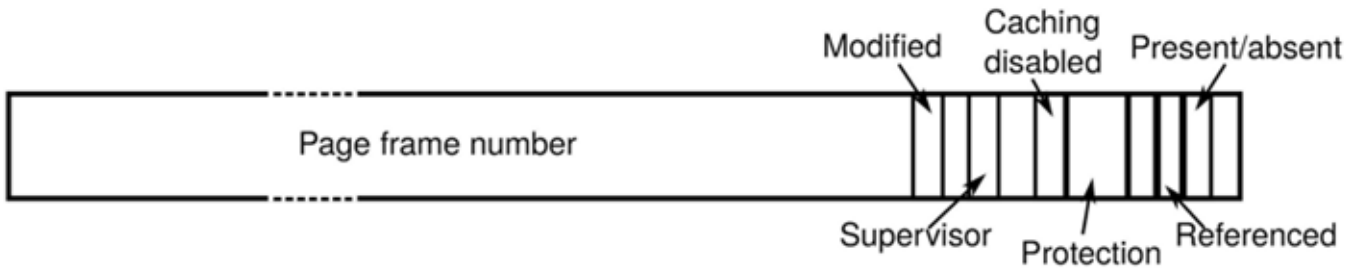


Figure 3.11 A typical page table entry.

- 4KB page size (typical)
- 32 bit systems:
  - 32 bit address space and up to 2^(32-12) page table entries (PTEs)
  - 4-byte per PTE, thereby up to 4 MB each process.
- 64 bit systems
  - Really: 48-bit or 57-bit address space.
  - Question: How many PTEs in if address space is 48 bit?

Page Table Sizes

A very large virtual address space would lead to a very large page table $\rightarrow$ not nice!

Possible solutions:

- Multi-level page tables
- Inverted page tables

Two-level Page Tables (x86)

Page tables are “walked” by hardware
MMU = Memory Management Unit

CR3 register
(special register to point to top of page table hierarchy)

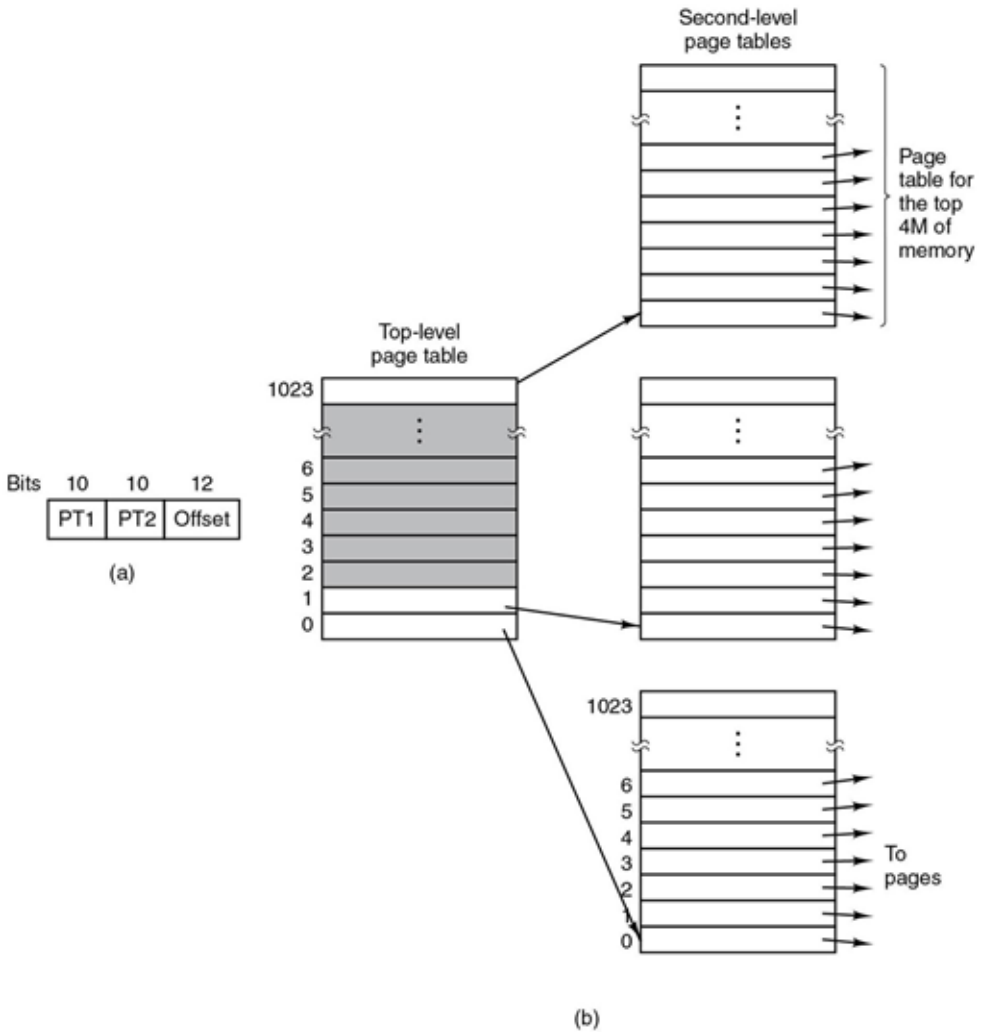
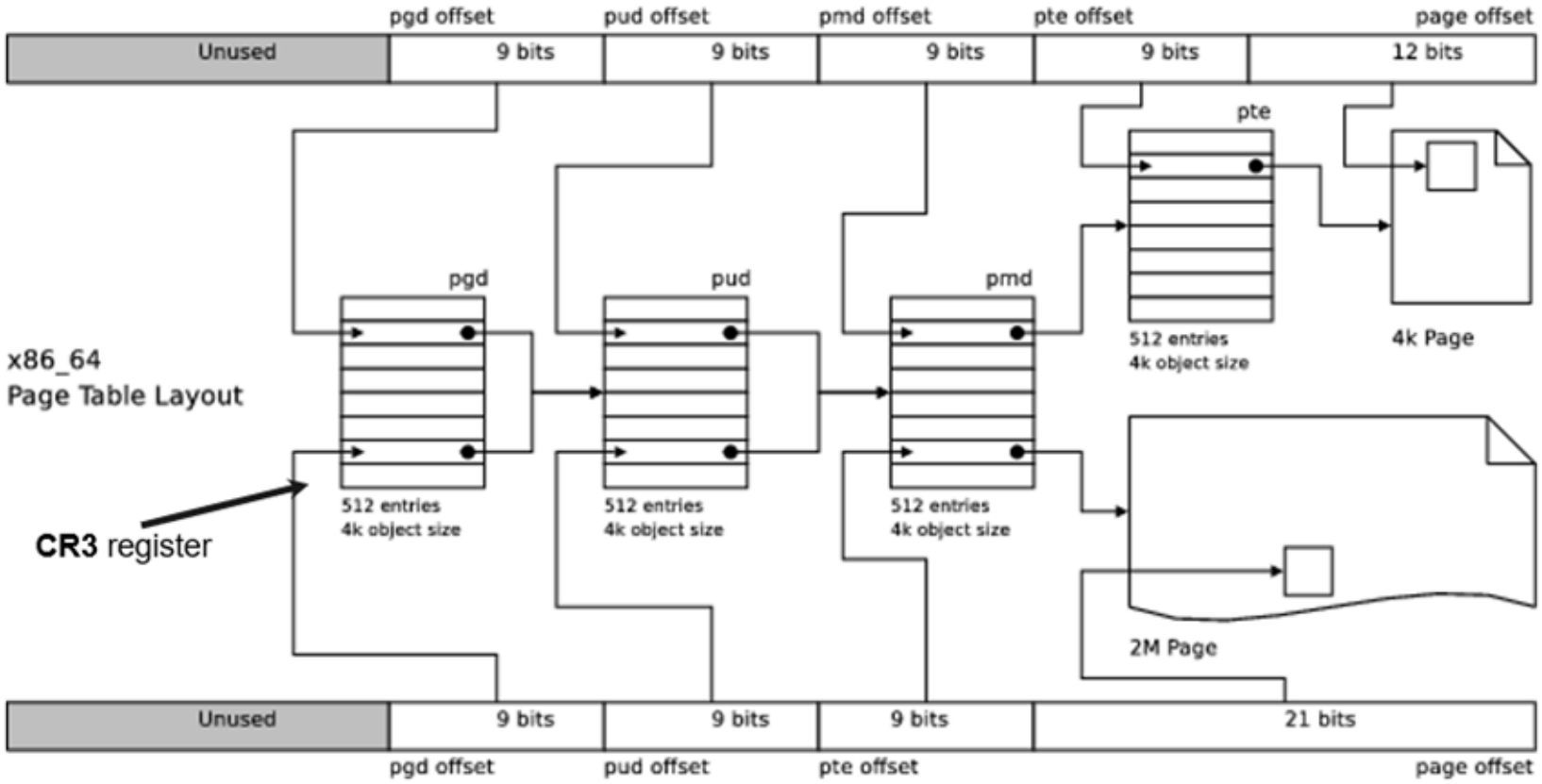


Figure 3.13 (a) A 32-bit address with two page table fields. (b) Two-level page tables.

– Question: How big is the table if three second-level page tables are active.

Four-level Page Tables (x86-64)



PGD: Page Global Directory, **PUD:** Page Upper Directory **PMD:** Page Mid-level Directory, **PTE:** page table entry

Inverted Page Tables (I A-64)

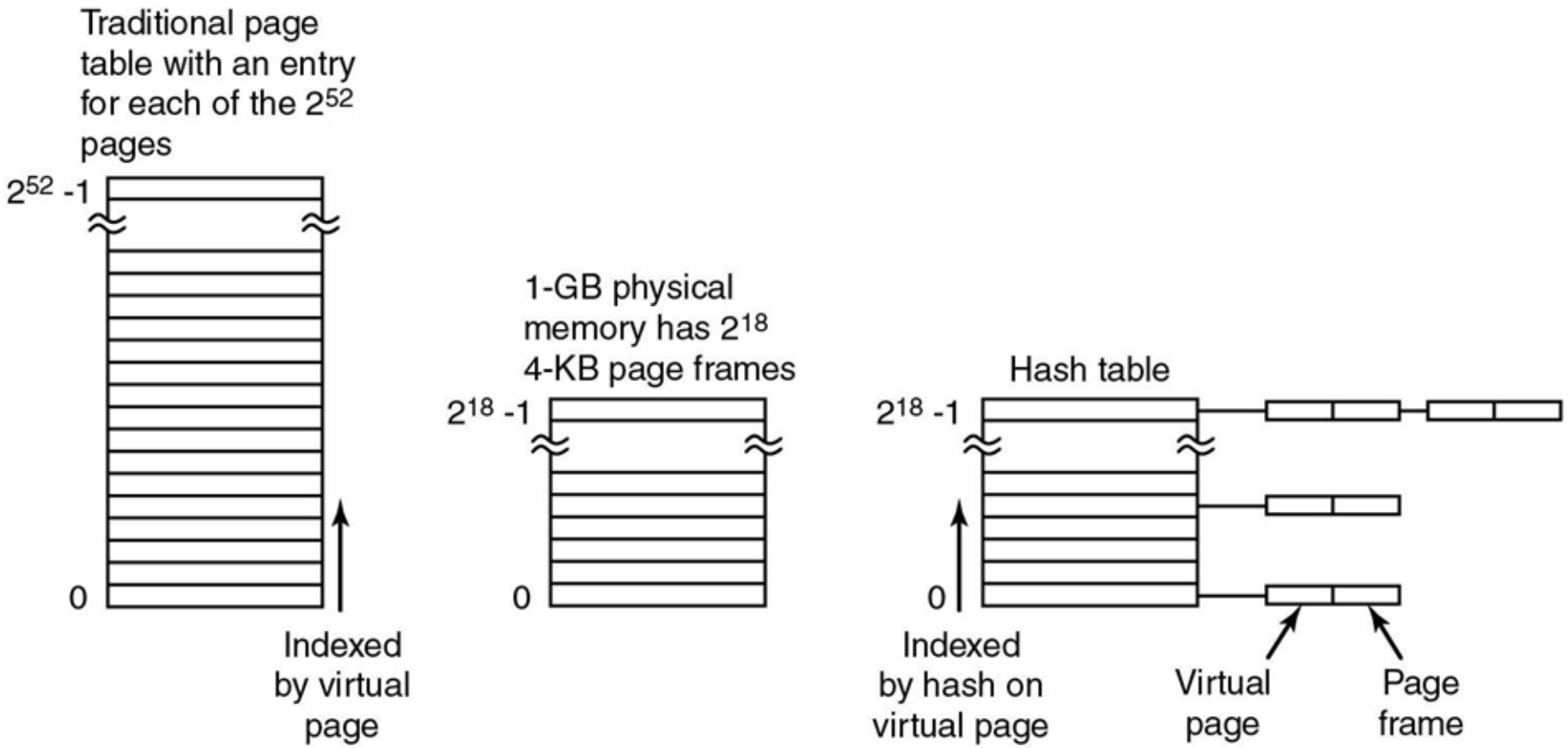
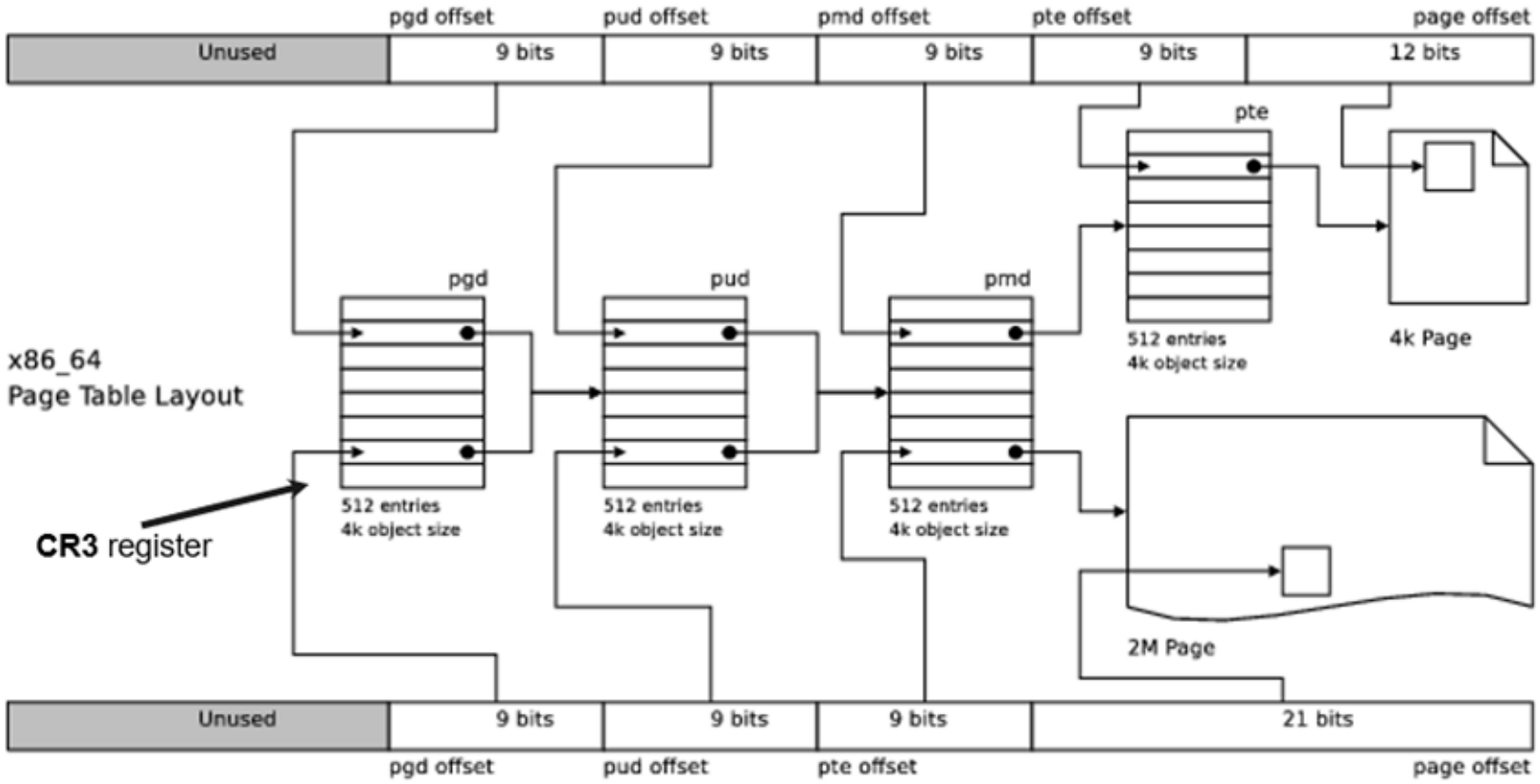


Figure 3.14 Comparison of a traditional page table with an inverted page table.

Four-Level Page Tables: Lots of Memory Accesses!



PGD: Page Global Directory, **PUD:** Page Upper Directory, **PMD:** Page Mid-level Directory, **PTE:** page table entry

Translation Lookaside Buffer (1)

Problem: MMU has to translate every single memory access

→ Performance problem

Solution: Cache previous translations, hoping programs

→ A high degree of locality.

Valid	Virtual Page	Modified	Protection	Page Frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Figure 3.12 A T L B to speed up page.

Translation Lookaside Buffer (2)

Extra material

Questions:

- How many T L B s?
- When to flush T L B?
- When to expect many T L B misses?

Translation Lookaside Buffer (3)

Extra material

Questions:

- **How many TLB s?**
- When to flush TLB?
- When to expect many TLB misses?

1. Super expensive memory: small number of entries (e.g., 64-128)
2. Sometimes separate entries for huge (2 MB) pages and normal (4 KB) pages
3. Often separate TLB s for code and data
4. 1 per CPU, but sometimes multiple levels of TLB (L1 TLB vs L2 TLB)

Translation Lookaside Buffer (4)

Extra material

Questions:

- How many TLB s?
- **When to flush TLB?**
- When to expect many TLB misses?

1. When entries change
2. Typically at context switch

Translation Lookaside Buffer (5)

Extra material

Questions:

- How many TLBs?
- When to flush TLB?
- **When to expect many TLB misses?**

1. After flushes
2. When the program exhibits little locality

TLB Misses and Page Fault Handling (1)

Software- vs. hardware-managed TLB:

- TLB misses handled by OS vs. hardware → Hardware walks PTs and fills TLB

  ↓

  E.g., OS may preload TLB entries that it expects to need later

- Flexibility vs. Efficiency

Copyright

This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.